

CS 635, Fall 2025

Module 5 Assignment

Prepared by:

Brandon Reynolds (821924175)

E-mail: bareynolds@sdsu.edu

San Diego State University

For Project 1, my partner and I did our best to keep everything organized which ultimately satisfied parts of the SOLID principles. Throughout the development of the Adaptive Library Management System (ALMS), my group focused on making sure we had solid design choices which in retrospect helped us adhere to the SOLID principles to keep the system organized and flexible as the project grew. This is reflected all the way from the arrangement of entities and transaction classes to the design of interfaces and the file persistence layer. While most of our project remained “solid,” there were areas that needed improvement which is where I updated the project.

The Single Responsibility Principle (SRP) states that each class should have only one reason to change. In our project, we designed the “entities” folder so that each file contained a single and focused class. For example, `book.py` handles book data and availability, `user.py` manages user information and borrowing records, and the transaction classes focus solely on borrowing and returning operations. This separation of concerns made the code easier to maintain and reduced the risk of accidental side effects when modifying one feature. As I reviewed the project, I also improved SRP compliance by reviewing responsibilities. One fix was for the original `Book` class since it mixed state management with the logic for checking in and out a book. By updating the typing it isolated the purpose of the `to_dict()` and `from_dict()` methods so that they were able to make the class primarily for books. With this change, I also updated the `Library` class so that it focused on managing collections of books and users with updated typing. Lastly, transaction logic was changed for the transaction classes to fit in for use in `library.py`. I felt these changes strengthened the SRP and improved the clarity of the overall design.

Next the Open–Closed Principle (OCP) emphasizes the idea that software entities should be open for extension but closed for modification. I found that the principle was first applied in our `library.py` structure since it allowed new types of transactions to be added without rewriting existing logic. For example, if we wanted to introduce a new transaction type such as a renewal or reservation we could implement it as a subclass rather than modify `library.py` itself. I was able to update the codebase to reinforce this idea through the way `Transaction` is implemented. Our base `Transaction` class defines the shared interface and behavior, while `BorrowTransaction` and

ReturnTransaction extend this functionality. Because the library interacts only with the abstraction rather than specific implementations this means new transaction behaviors can be added with minimal changes to the system. This led me to change the hard coded imports inside library.py methods in order to limit how dependencies are used so that the system is more flexible and compliant with OCP.

The Liskov Substitution Principle (LSP) ensures that subclasses can be substituted in place of their base classes without breaking the program. Our project demonstrated strong LSP compliance through the Transaction class. Both BorrowTransaction and ReturnTransaction behave consistently as Transaction objects and can be passed into the same workflows without any special logic. With our test suites, it reaffirmed me that each subclass follows the behavior defined in the abstract base by allowing them to be interchanged seamlessly. LSP was also reflected in the implementation of the Searchable interface which is used by both Book and User. Because any class implementing Searchable supports the matches() method, the system can perform uniform search operations without needing to differentiate between entity types.

The Interface Segregation Principle (ISP) states that clients should not be forced to depend on methods they do not use. In our ALMS project, we adhered to ISP by designing a lean and focused Searchable interface that defines only one essential method of matches(). Both the Book and User classes use this method naturally which allows them to participate in search operations without taking on unnecessary extra methods. This avoided the creation of large and bloated interfaces while helping to keep the architecture clean. If our system were to be expanded more, I could introduce additional interfaces to handle more specific responsibilities. I feel that in its current state, the project demonstrates strong compliance with ISP since it uses tightly scoped interfaces.

Finally, the Dependency Inversion Principle (DIP) emphasizes that high level modules should depend on abstractions rather than concrete implementations. In our original project, the Library class directly imported specific transaction methods inside method bodies, which created hidden dependencies and reduced flexibility. After revising the methods, the library file now clearly shows the abstraction of the classes

which improves extensibility. It is also followed in the “test_suites” folder since the tests rely on interfaces and mock dependencies. Overall I think our project was able to meet most of DIP which helps it be easily maintainable and scalable.

In conclusion, my group’s ALMS project demonstrated a strong initial attempt at applying the SOLID principles. After carefully reviewing it, I was able to make meaningful refinements that improved the design further. I was able to give each class a more clearly defined role, keep the extension of core components without modification alive, and make sure that the transaction processes and entity interactions follow clear abstractions through updating typing. Some minor changes were made to the testing suite which helped to reaffirm the principles were being followed. So while I didn’t make a major restructure, I believe these updates create a codebase that is more maintainable and scalable which is exactly the goals emphasized by the SOLID principles. In the end, I felt that reviewing and applying these principles helped to solidify my project for the long term.

Testing to verify updated code still performs as expected:

The image displays two screenshots from a development environment. The left screenshot shows a code editor with a JSON file named 'alms_data.json'. The JSON contains two arrays: 'books' and 'users'. The 'books' array has two entries: one with isbn '111' (Clean Code by Robert Martin, 2 total copies, 2 available) and another with isbn '222' (The Pragmatic Programmer by Andrew Hunt, 1 total copy, 1 available). The 'users' array has two entries: 'u1' (Alice, alice@example.com) and 'u2' (Bob, bob@example.com), both with empty 'borrowed_isbns' lists. The right screenshot shows a terminal window with the output of a pytest command. It reports that 17 tests passed in 0.03 seconds. The tests are categorized by file: test_entities.py (52%), test_persistence.py (70%), and test_transactions.py (100%). Below the terminal output, a snippet of test results is visible, showing the execution of 'Borrow' and 'Return' actions for book '111' by user 'u1', confirming the state of available copies.

```

1 {
2   "books": [
3     {
4       "isbn": "111",
5       "title": "Clean Code",
6       "author": "Robert Martin",
7       "total_copies": 2,
8       "available_copies": 2
9     },
10    {
11      "isbn": "222",
12      "title": "The Pragmatic Programmer",
13      "author": "Andrew Hunt",
14      "total_copies": 1,
15      "available_copies": 1
16    }
17  ],
18  "users": [
19    {
20      "user_id": "u1",
21      "name": "Alice",
22      "email": "alice@example.com",
23      "borrowed_isbns": []
24    },
25    {
26      "user_id": "u2",
27      "name": "Bob",
28      "email": "bob@example.com",
29      "borrowed_isbns": []
30    }
31  ]
32 }

```

```

(base) brandonreynolds@randons-Laptop adaptive-library-system % pytest
===== test session starts =====
platform darwin -- Python 3.10.9, pytest-7.1.2, pluggy-1.0.0
rootdir: /Users/brandonreynolds/CS-635-Assignments/adaptive-library-system, configfile: pytest.ini, testpaths: test_suites
plugins: anyio-3.5.0
collected 17 items

test_suites/test_entities.py ..... [ 52%]
test_suites/test_persistence.py ... [ 70%]
test_suites/test_transactions.py .... [100%]

===== 17 passed in 0.03s =====
(base) brandonreynolds@randons-Laptop adaptive-library-system %

```

```

test_suites/test_entities.py .....
test_suites/test_persistence.py ...
test_suites/test_transactions.py ....

=====
(base) brandonreynolds@randons-Laptop adaptive-library-system %
===== Books =====
111 Clean Code Robert Martin 2
222 The Pragmatic Programmer Andrew Hunt 1

Borrow '111' for user 'u1'
Available copies for 111: 1

Return '111' for user 'u1'
Available copies for 111: 2
(base) brandonreynolds@randons-Laptop adaptive-library-system %

```